

Conceptos por capítulo

Capítulos 04 al 08 · Lista de referencia rápida para el examen teórico

Cómo está organizado: los conceptos siguen la tabla de cobertura oficial del banco de 50 preguntas — Cap. 04 (preguntas 1-10), Cap. 05 (11-20), Cap. 06 (21-30), Cap. 07 (31-40) y Cap. 08 (41-50). Cuando un concepto se enseña en un capítulo pero el examen lo pregunta en otro, está indicado. El examen está redactado sobre ParkFlow 360 (Cliente 1:N Vehículo); el equivalente en FitClub es Socio 1:N Membresía.

CAPÍTULO 04 — Fundamentos web, HTTP, REST, Spring MVC, DTOs y validación

1. Cliente, servidor y protocolo

Cliente

El software que inicia la petición: la app web en React, la app móvil o Postman. No es la persona, es el programa.

Servidor

El backend en Spring Boot que recibe la petición, la procesa y responde. Es el único que toca directamente la base de datos.

HTTP

El protocolo de comunicación por turnos entre cliente y servidor: uno pide, el otro responde, y la conversación termina ahí.

HTTPS

El mismo HTTP pero cifrado. La "S" es de *Secure*: protege los datos de quien pueda interceptarlos en el camino.

URL / URI

La dirección única que identifica un recurso, por ejemplo `/api/socios/7`.

Método HTTP

El verbo que expresa la intención de la petición: GET, POST, PUT, PATCH o DELETE.

Headers

Metadatos de la petición o la respuesta, no del negocio: `Content-Type` (formato del cuerpo), `Accept` (formato esperado), `Authorization` (quién eres).

Body

El contenido real que se envía o se recibe, normalmente en JSON. Se usa en escrituras (POST, PUT); en un GET no corresponde.

Parámetros

De ruta, cuando identifican el recurso (`/socios/7`); de consulta, cuando filtran, ordenan o paginan (`?estado=activo`).

JSON

El formato de texto en que viajan los datos: pares clave-valor entre llaves. Es la representación favorita de una API REST.

2. REST y la API

REST

Estilo arquitectónico, no un protocolo: organizar todo en torno a recursos, usar siempre los verbos HTTP estándar, mantener una interfaz uniforme y no guardar estado entre peticiones.

Recurso

La "cosa" del negocio sobre la que se opera, expresada como sustantivo: socios, membresías. Nunca como acción (`/crearSocio` es un error de diseño).

API REST

El contrato completo que el servidor expone: qué operaciones existen, qué datos se envían, qué se devuelve, qué errores pueden ocurrir y en qué formato.

Endpoint

Una operación concreta dentro de esa API: la combinación de un método HTTP y una ruta. `POST /api/socios` es un endpoint.

Stateless (sin estado)

El servidor no recuerda peticiones anteriores. Cada petición llega completa y se explica sola. Por eso la verdad vive en la base de datos, no en la memoria del proceso.

GET

Consultar sin modificar. Es seguro (no cambia nada) e idempotente.

POST

Crear un recurso nuevo. **No es idempotente:** repetirlo crea duplicados. Es la causa práctica de los registros repetidos.

PUT

Crear o reemplazar por completo el recurso de una URI conocida. Es idempotente aunque a veces cree.

PATCH

Modificar solo algunos campos de un recurso existente. No está garantizado como idempotente.

DELETE

Eliminar el recurso indicado. Es idempotente: repetirlo no borra "más".

Idempotencia

Propiedad por la cual ejecutar la misma petición una o muchas veces deja el sistema en el mismo estado final.

Códigos HTTP

Número de tres dígitos que resume el resultado. 2xx: salió bien. 4xx: el problema es del cliente. 5xx: el problema es del servidor.

200 OK

Operación exitosa con contenido en la respuesta. Típico de un GET.

201 Created

Se creó un recurso nuevo. Es la respuesta natural de un POST de registro; acompaña la cabecera `Location`.

204 No Content

Éxito sin cuerpo que devolver. Típico de un DELETE.

400 Bad Request

La petición está mal formada: falta un campo obligatorio, el formato es inválido, el JSON está roto.

404 Not Found

La petición está bien escrita, pero el recurso identificado no existe.

3. Spring, Spring Boot y Spring MVC

Spring Framework

El contenedor que crea, ensambla y entrega los objetos de la aplicación, para que ninguna clase tenga que fabricarse sus propios colaboradores.

Spring Boot

Spring ya preconfigurado y con servidor embebido (Tomcat): arranca la aplicación sin que tengas que montar la infraestructura a mano.

Spring MVC

El módulo web que recibe cada petición HTTP y decide qué método de qué Controller debe atenderla. Traduce entre HTTP y objetos Java; no contiene reglas de negocio.

DispatcherServlet

La puerta única por la que entra toda petición antes de repartirse (patrón *Front Controller*).

@RestController

Marca la clase como puerta HTTP que devuelve datos en JSON, no vistas. Equivale a `@Controller` + `@ResponseBody`.

@RequestMapping

Ruta base de la clase (`/api/socios`). Si se usa en un método sin indicar `method=`, mapea todos los verbos: por eso se prefieren los atajos.

@GetMapping / @PostMapping

Atajos que atan un método a un verbo concreto y hacen explícita la intención semántica del endpoint.

@PathVariable

Lee un valor de la ruta. Se usa cuando ese dato *identifica* el recurso.

@RequestParam

Lee un parámetro de la query string. Se usa para filtrar, ordenar, paginar o para datos opcionales.

@RequestBody

Lee el cuerpo JSON de la petición y lo convierte en el Request DTO.

ResponseEntity

Permite decidir con precisión el código HTTP, las cabeceras y el cuerpo exacto de la respuesta.

Nota: IoC, DI, Component Scan y Bean se enseñan aquí, pero el examen los pregunta en el Capítulo 06, ligados a la arquitectura hexagonal. Están listados en ese capítulo.

4. DTOs y validación de formato

DTO (Data Transfer Object)

Objeto que define exactamente qué datos entran o salen por la API. Existe para que el contrato de la API no dependa de la base de datos.

Request DTO

Define lo que el cliente *puede* enviar. Deja fuera el id generado, las fechas de auditoría y el estado interno.

Response DTO

Define lo que se expone hacia afuera. Permite ocultar campos sensibles y mantener estable el contrato aunque el dominio cambie.

record

Tipo de Java inmutable: se llena una sola vez y no se modifica. Ideal para DTOs y para ApiError.

Jackson

La librería que traduce entre JSON y objetos Java: deserializa al entrar, serializa al salir.

Deserialización / Serialización

Entrada: JSON → Request DTO. Salida: Response DTO → JSON.

Jakarta Validation

Las reglas declarativas que se escriben sobre el DTO: `@NotBlank`, `@NotNull`, `@Email`, `@Size`, `@Pattern`.

@Valid

Colocada en el parámetro del Controller, dispara esa validación antes de ejecutar cualquier lógica de negocio.

Validación de formato

La primera barrera: ¿está bien escrito? La ejecuta el Controller y no consulta nada. Si falla, responde 400.

5. Herramientas cliente y de desarrollo

Postman

Cliente externo para probar la API enviando peticiones reales, sin necesidad de frontend.

IntelliJ HTTP Client

Lo mismo pero dentro del IDE, en archivos `.http` que se versionan junto al código.

Maven

Gestor de dependencias y construcción: descarga las librerías declaradas en el `pom.xml` y compila el proyecto.

Git

Control de versiones: registra cada cambio, quién lo hizo y cuándo.

Debugger

Permite pausar la ejecución y ver el estado real de cada variable en ese instante.

Breakpoint

El punto donde el debugger detiene la ejecución.

Stack trace

El informe de la cadena de llamadas que llevó al error. Va al log del servidor, nunca al cliente.

CAPÍTULO 05 — PostgreSQL, JDBC, ORM, JPA, Hibernate y Spring Data JPA

1. La cadena de persistencia

ORM (Object-Relational Mapping)

La técnica de traducir automáticamente entre objetos Java y filas de tablas. Resuelve el desajuste entre ambos mundos, pero **no elimina** la necesidad de saber SQL ni de diseñar bien la base.

JPA

La **especificación** (Jakarta Persistence): define las anotaciones y el contrato de persistencia. Por sí sola no ejecuta nada.

Hibernate

La **implementación** más usada de JPA: es quien genera y ejecuta el SQL real, gestiona el contexto de persistencia y detecta los cambios.

Spring Data JPA

Capa de **conveniencia** sobre JPA: crea los repositorios a partir de interfaces y deriva consultas del nombre del método.

JDBC

La API de bajo nivel de Java que realmente envía el SQL a la base de datos. Todo lo anterior se apoya en ella.

PostgreSQL

El motor relacional donde viven las tablas, las restricciones, los índices y los datos reales.

JpaRepository

La interfaz de Spring Data con el CRUD listo. Es infraestructura, **no** es el modelo de dominio.

Frase clave para la defensa: JPA es la norma, Hibernate la cumple, Spring Data la simplifica, JDBC conecta y PostgreSQL guarda. Cada uno está en un nivel distinto; no son sinónimos.

2. Mapeo de entidades

@Entity

Declara la clase como tipo gestionado por JPA. Registra la *clase* en el metamodelo; son sus *instancias* las que entran al contexto de persistencia.

@Table

Indica la tabla física y, si hace falta, el schema: `@Table(name="socio", schema="fitclub")`.

@Id

Señala el atributo que representa la clave primaria; es la identidad de la fila.

@GeneratedValue

Define quién genera el valor del id. `IDENTITY` delega en la columna autoincremental; `SEQUENCE` usa una secuencia explícita.

@Column

Describe la columna real: `name`, `nullable`, `length`, `unique`. Es el puente entre el nombre Java y el nombre SQL.

SocioJpaEntity ≠ Socio de dominio

Son clases distintas porque cambian por motivos distintos: la entidad cambia si cambia la tabla; el dominio cambia si cambia una regla del negocio.

3. Configuración y verificación

Schema

Una sección con nombre **dentro** de la base de datos que agrupa tablas. No es lo mismo que la base: puedes estar conectado a la base correcta y apuntar al schema equivocado.

hibernate.ddl-auto=validate

Al arrancar, Hibernate solo verifica que el mapeo calce con la base. No modifica nada. Comprueba tablas, columnas y tipos, pero **no** valida FKs, CHECK, índices ni UNIQUE.

create / create-drop

Recrean el esquema al arrancar (y create-drop además lo borra al cerrar). Destruyen datos, índices y restricciones: nunca fuera de tests.

open-in-view=false
Cierra el contexto de persistencia al salir de la capa de aplicación, obligando a resolver la carga perezosa dentro del caso de uso.

Contexto de persistencia
La "sesión" donde Hibernate mantiene las entidades gestionadas durante la transacción.

INSERT
La sentencia SQL que agrega la fila. Con JPA no se envía al llamar a `save`, sino en el *flush* del commit.

DataGrip / pgAdmin
Cliente de base de datos para comprobar con un SELECT que la fila realmente existe, de forma externa a tu código.

Logs SQL
Con `show-sql` activado, muestran el SQL que Hibernate ejecutó de verdad. Un INSERT en el log no implica que hubo commit.

CAPÍTULO 06 — Arquitectura hexagonal, IoC/DI, puertos, casos de uso y patrones

1. Las capas

Arquitectura hexagonal
Aislar el núcleo de negocio de toda la tecnología: la base de datos, el framework web y el formato JSON son mecanismos intercambiables, no el corazón del sistema.

Regla de dependencias
Las flechas apuntan siempre hacia adentro: la infraestructura conoce al núcleo, el núcleo nunca conoce a la infraestructura.

Dominio
El corazón: entidades, reglas e invariantes del negocio. Sin frameworks, sin anotaciones, sin base de datos. Responde a "¿qué es válido?".

Capa de aplicación
Los casos de uso: orquestan los pasos de una operación y marcan la frontera transaccional. Responde a "¿qué pasos tiene registrar un socio?".

Infraestructura
Todo lo que toca el exterior: Controllers, adaptadores de persistencia, entidades JPA, configuración. Responde a "¿cómo entra y sale la información?".

2. Puertos y adaptadores

Port IN
Interfaz del núcleo que declara una **capacidad** del sistema (`RegistrarSocioUseCase`), en lenguaje de negocio. No menciona HTTP, JSON ni Spring Data, para poder invocarse desde cualquier canal.

Port OUT
Interfaz que declara lo que el núcleo **necesita** del exterior (`SocioRepositoryPort`). No debe extender `JpaRepository`: eso metería la persistencia dentro del negocio.

Adapter IN
Traduce del mundo exterior al núcleo. Ejemplo: el `SocioController`, que convierte una petición HTTP en una llamada al Port IN.

Adapter OUT
Implementa el Port OUT contra una tecnología concreta. Ejemplo: el `SocioPersistenceAdapter`, que sí conoce Hibernate y la entidad JPA.

Application Service / Caso de uso
Orquesta la operación completa: aplica las reglas, invoca los puertos de salida y define el límite transaccional. Depende de abstracciones, nunca de `SpringDataRepository`.

Controller ≠ Port IN
El puerto dice *qué* se puede pedir; el Controller traduce *cómo* llegó ese pedido. El Controller es reemplazable, el puerto es estable.

3. Inversión de control e inyección

IoC (Inversión de Control)
El framework crea y gestiona los objetos, no tu código con `new`.

DI (Inyección de Dependencias)
La técnica con la que el contenedor entrega a cada clase los colaboradores que necesita.

Inyección por constructor
La forma preferida: las dependencias quedan obligatorias, explícitas y los campos pueden ser `final`. Permite probar la clase pasándole un doble de prueba.

Component Scan
El barrido que busca las clases anotadas y las registra en el contenedor.

Bean
Cualquier objeto creado y gestionado por el contenedor de Spring. Se marcan con `@Service`, `@Repository` o `@RestController`.

Inversión de dependencias (DIP)
El principio: quien necesita el servicio define la interfaz; quien la cumple, la implementa. DIP dice hacia dónde apuntan las flechas; IoC y DI son la herramienta que lo hace posible en ejecución.

4. Patrones

Repository Pattern
Tratar la persistencia como si fuera una colección de objetos de dominio, ocultando por completo cómo se almacenan.

Adapter Pattern

Una pieza que traduce entre dos interfaces que no encajan: la que el núcleo pide y la que la tecnología ofrece.

Mapper Pattern

Traducir un modelo en otro entre capas, sin acoplarlos.

Mapper

El traductor concreto. Hacen falta dos, uno por frontera: Web DTO ↔ Dominio (en el adaptador de entrada) y Dominio ↔ JPA Entity (en el de salida). Viven en los adaptadores, nunca en el dominio.

CAPÍTULO 07 — Relación 1:N entre módulos (Socio 1:N Membresía)

Cardinalidad 1:N

Un socio puede tener varias membresías, y cada membresía pertenece a un solo socio. El Socio es la entidad padre; la Membresía, la dependiente.

PK (clave primaria)

Identifica de forma única cada fila. Es única, no nula y, por convención de diseño, estable en el tiempo.

FK (clave foránea)

Columna que referencia la clave de otra tabla. **Vive siempre en el lado "muchos":** `socio_id` está en la tabla de membresías.

Integridad referencial

La garantía del motor de que ninguna FK apunte a una fila inexistente. Impide registros huérfanos y borrados que romperían la relación.

UNIQUE

Restricción que impide valores repetidos en una columna. Es la barrera final ante duplicados, porque el motor la aplica con control de concurrencia.

@ManyToOne

Declara la cardinalidad desde el lado dependiente: muchas membresías apuntan a un socio. Es mapeo, **no** es la FK física.

@JoinColumn

Indica qué columna guarda esa referencia: `@JoinColumn(name="socio_id")`.

FetchType.LAZY

Cargar la asociación solo si se usa. En `@ManyToOne` hay que declararlo explícitamente porque **por defecto es EAGER**. Evita el problema N+1 en listados.

LazyInitializationException

El error que aparece al tocar un dato perezoso fuera del contexto de persistencia.

Los tres niveles de la relación

Dominio: la Membresía guarda solo `socioId`, un dato simple. JPA: la conoce como objeto, con `@ManyToOne`. PostgreSQL: es una columna con una restricción física. Ninguno reemplaza a los otros.

Colaboración entre módulos

Un módulo consulta a otro por su **caso de uso** (`ConsultarSocioUseCase`), nunca leyendo sus tablas ni su repositorio. Así se evita acoplarse al esquema ajeno y saltarse sus reglas.

Validar existencia en el service

El caso de uso comprueba que el socio exista antes de guardar, aunque la FK ya proteja el dato: la FK llega tarde y habla en idioma de base de datos; el service da un 404 claro y a tiempo.

CAPÍTULO 08 — Errores profesionales, validaciones de negocio y transacciones

1. Manejo de errores

Excepción de negocio

Expresa qué regla del dominio se rompió (`SocioNoEncontradoException`, `DocumentoDuplicadoException`). **No conoce HTTP** ni `ResponseEntity`: la misma regla debe valer por HTTP, por lotes o por CLI.

@RestControllerAdvice

Manejador global: un único lugar donde se traducen todas las excepciones a respuestas HTTP. Evita repetir `try/catch` en cada Controller y garantiza una política uniforme.

@ExceptionHandler

El método que atiende un tipo concreto de excepción y decide su código HTTP y su cuerpo.

ApiError

El formato fijo de toda respuesta de error: fecha, código, tipo, mensaje y ruta. Permite al frontend manejar errores con una sola función y evita filtrar detalles internos.

Capturar Exception genérica

Mala práctica: iguala errores que deben distinguirse, oculta bugs reales y desincroniza la política de errores. Lo genérico va una sola vez en el advice, con la traza al log.

2. Los tres niveles de validación

Validación de formato

En la frontera, sobre el DTO, con Jakarta Validation. ¿Está bien escrito? Si falla → **400**.

Validación de negocio

En el caso de uso. ¿Tiene sentido según las reglas? Normalmente consulta el estado del sistema. Si falla → **409** o **404**.

Integridad física

En PostgreSQL, con PK, FK, UNIQUE, NOT NULL y CHECK. Es la última barrera y la única válida para todos los accesos, incluidos los que no pasan por tu API.

Defensa en profundidad

Las tres barreras encadenadas. No son redundantes: fallan por motivos distintos y solo la base resuelve las condiciones de carrera.

3. Semántica de los códigos de error

400 Bad Request

La petición es inválida en su forma. Reintentarla igual volverá a fallar: el cliente debe corregir.

404 Not Found

La petición es correcta pero el recurso identificado no existe. Es una respuesta normal, no un fallo del sistema.

409 Conflict

La petición es correcta pero choca con una regla o con el estado actual: documento duplicado, transición no permitida. Es el código propio de las violaciones de reglas de negocio.

500 Internal Server Error

El cliente hizo todo bien y el servidor falló. Un 500 evitable indica una excepción no manejada, es decir, un defecto de diseño.

DataIntegrityViolationException

La excepción que Spring lanza cuando la base rechaza la fila. Debe traducirse a **409**, no dejarse caer en un 500.

4. Transacciones

Transacción

Unidad de trabajo indivisible: o se confirma todo (commit) o no se confirma nada (rollback). Evita estados intermedios incoherentes.

Atomicidad

La propiedad de "todo o nada". Junto con consistencia, aislamiento y durabilidad forma el conjunto **ACID**.

@Transactional

Delimita esa unidad de trabajo de forma declarativa: Spring abre la transacción al entrar al método, confirma si termina bien y revierte si se propaga una excepción.

Frontera transaccional

Se ubica en el **caso de uso**, nunca en el Controller: la operación de negocio es lo que debe ser atómico, y la atomicidad no puede depender del canal por el que entró la petición.

TRANSVERSAL — El flujo end-to-end

Recorrido completo de una petición

Cliente HTTP → Spring MVC enruta → Controller recibe con `@PostMapping` y `@RequestBody` → Jackson deserializa al Request DTO → `@Valid` valida el formato → el mapper traduce al dominio → el Controller invoca el Port IN → el caso de uso abre la transacción y aplica las reglas → llama al Port OUT → el adaptador mapea a la entidad JPA → Spring Data e Hibernate generan el SQL → JDBC lo ejecuta contra PostgreSQL → la base valida las restricciones y confirma → el resultado regresa como Entity → Dominio → Response DTO → JSON → el Controller responde **201 Created**.

Flujo de error

La excepción se lanza donde se detecta el problema, sube sin traducirse, la transacción se revierte, y un único lugar —el `@RestControllerAdvice`— decide el código HTTP y construye el `ApiError`.